Vietnam National University HCMC

University of Science

Faculty of Information Technology

Advanced Program in Computer Science

The Code Cup

Midterm Project Report

CS426 — Mobile Device Application Development

Student:

Trinh Vo Nam Kiet

Student ID: 24125013
Class: 24A01

July 29, 2026



Contents

1 Introduction **2**

1.1 Technology Stack . . . 2
1.2 Key Features . . . 2
1.3 New Features . . . 2
1.4 Lifecycle Management . . . 3

2 Architecture **3**

2.1 Layer Diagram . . . 3
2.2 Layer Responsibilities . . . 4

3 Screens **4**

3.1 Home Screen . . . 4
3.2 Details Screen . . . 5
3.3 My Cart Screen . . . 5
3.4 Order Success Screen . . . 6
3.5 My Orders Screen . . . 6
3.6 Rewards Screen . . . 7
3.7 Redeem Rewards Screen . . . 8
3.8 Profile Screen . . . 8

4 Domain Model **9**

4.1 Entity Overview . . . 9
4.2 Entity Details . . . 10

5 Business Rules Implementation **10**

6 Testing **11**

6.1 Test Summary . . . 11
6.2 Unit Tests . . . 11
6.3 Integration Tests . . . 12
6.4 UI (Widget) Tests . . . 13

7 Conclusion **13**

1 Introduction

The Code Cup is a mobile coffee shop application built with Flutter, targeting Android. Users can browse a coffee menu, customize and order drinks, collect loyalty stamps, earn and redeem reward points, and manage their profile.

1.1 Technology Stack

Technology	Purpose	Rationale
Flutter (Dart)	Cross-platform UI framework	Write once, deploy to mobile, web, and desktop from a single codebase
Android	Target platform	Required by specification
MVVM	Architecture pattern	Clean separation of concerns; UI and logic can be developed and tested independently
ChangeNotifier	State management	Built-in Flutter mechanism; pairs naturally with MVVM via reactive listeners
Hive	Persistence	Lightweight, fast, pure Dart key-value store with no native dependencies
flutter_test	Testing	Native Flutter framework, no extra dependencies required

Table 1: Technology stack with rationale

1.2 Key Features

- **Product browsing:** Browse a grid of coffee products with images and names.
- **Drink customization:** Customize each drink by shot type, size, and ice level, with real-time price updates.
- **Shopping cart:** Add, view, and remove items before checkout.
- **Loyalty stamps:** Earn one stamp per order (max 8), then reset for rewards.
- **Reward points:** Earn points based on order value and redeem them for free items.
- **Order tracking:** View ongoing and past orders with swipe-to-toggle status.
- **Profile management:** Edit name, email, phone, address, and avatar with inline editing.
- **Data persistence:** All data persists across app restarts via Hive.

1.3 New Features

- **Edit Cart Item:** Cart line items are tappable. Tapping opens the Details screen pre-filled with the item's current customizations and quantity.
- **Centered Add-to-Cart Notification:** "Added to cart" shows as a centered card overlay instead of a bottom snackbar, auto disappears after 1 second.

1.4 Lifecycle Management

The app persists runtime state across Flutter lifecycle transitions to prevent data loss. On each lifecycle state change (`paused`, `inactive`, `detached`, `hidden`), the current cart items, orders, and loyalty data are saved to Hive immediately. On app resume (`resumed`), the persisted state is restored and viewmodels are refreshed.

- **Edit Cart Item:** Cart line items are tappable. Tapping opens the Details screen pre-filled with the item's current customizations and quantity.
- **Centered Add-to-Cart Notification:** "Added to cart" shows as a centered card overlay instead of a bottom snackbar, auto disappears after 1 second.

2 Architecture

The application follows the **Model–View–ViewModel (MVVM)** pattern with manual dependency injection (no DI framework).

2.1 Layer Diagram

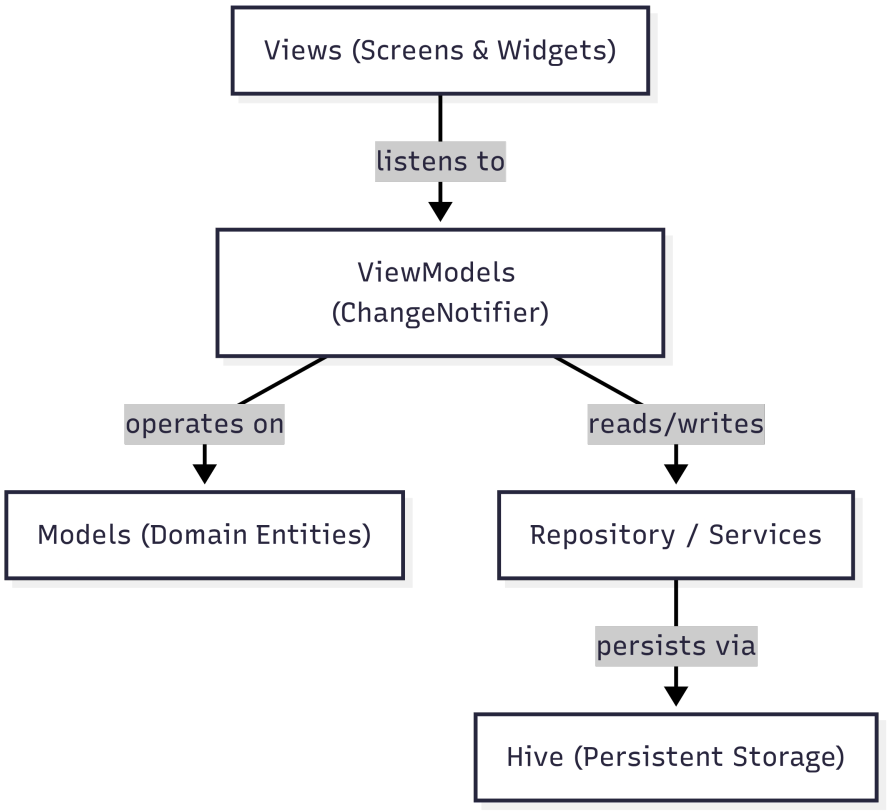


Figure 1: Layer Diagram

2.2 Layer Responsibilities

Models (`lib/models/`)

Pure Dart domain entities with no framework dependencies. Business logic lives in entity methods (e.g. `Order.calculatePointsEarned()`, `CartItem.calculateSubtotal()`). Constructors use `assert` for invariant validation.

ViewModels (`lib/viewmodels/`)

State holders extending `ChangeNotifier`. Each screen has a corresponding ViewModel that manages its state and exposes methods for user actions. ViewModels are manually wired in `main.dart` and passed as constructor parameters.

Views (`lib/views/`)

`StatelessWidget` screens that receive their ViewModel via constructor injection. Navigation uses raw `Navigator.push`.

Repository (`lib/repositories/`)

`AppRepository` aggregates all ViewModels and coordinates cross-cutting concerns (e.g. awarding stamps and points after checkout).

Services (`lib/services/`)

`LocalStorage` wraps `Hive` for persisting user profile data, orders, cart items, and loyalty stamp count.

3 Screens

The application consists of 8 screens, each mapped to a specification requirement.

3.1 Home Screen

Displays a greeting header, the user's loyalty card, and a grid of available coffee products. The bottom navigation bar provides access to all main sections.

Interactions:

- Tap a product card → Navigate to Details Screen
- Bottom navigation tabs → Navigate to corresponding screen

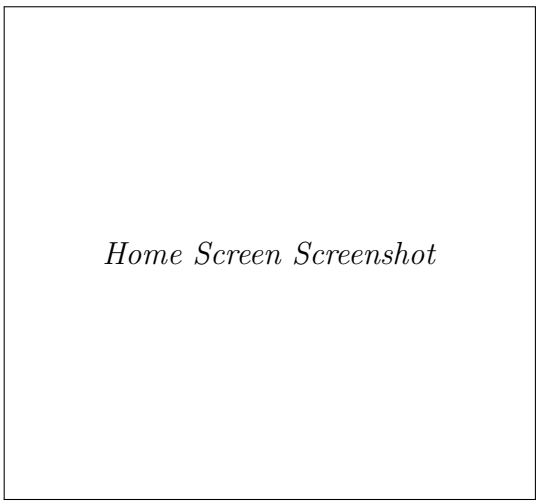


Figure 2: Home Screen

3.2 Details Screen

Shows product details with customization options (shot type, size, ice level). Total price updates reactively as options and quantity change.

Interactions:

- Cart icon → Navigate to My Cart
- "Add to Cart" button → Persist item, show centered overlay confirmation
- Back button → Return to Home

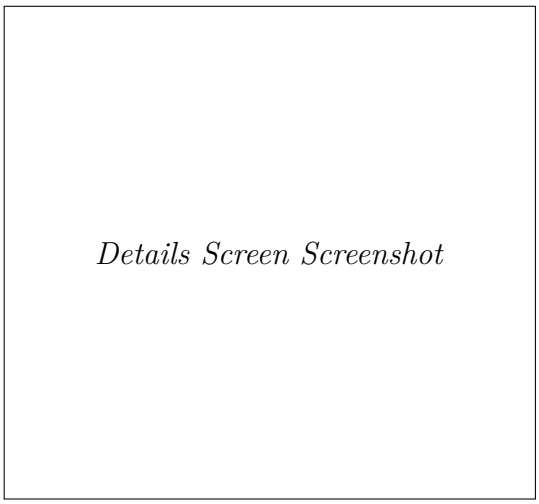


Figure 3: Details Screen with Customization

3.3 My Cart Screen

Renders cart items in a list view reflecting selected customizations. Displays the aggregate total price.

Interactions:

- "Checkout" button → Navigate to Order Success
- Swipe left on item → Remove from cart

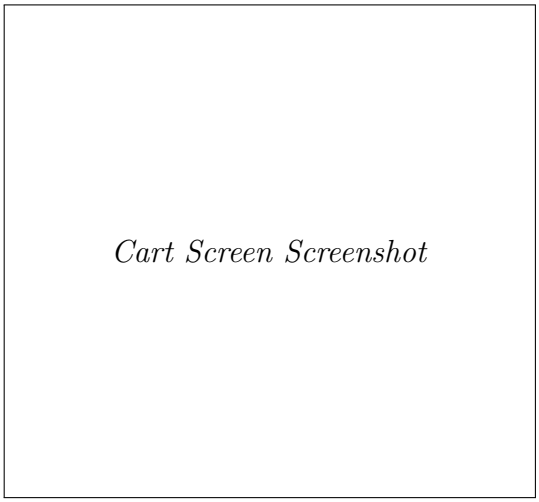


Figure 4: My Cart Screen

3.4 Order Success Screen

Confirms the order was placed successfully.

Interactions:

- "Track My Order" button → Navigate to My Orders

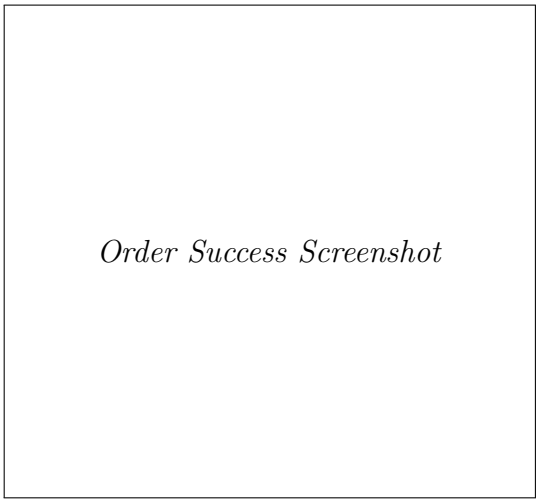


Figure 5: Order Success Screen

3.5 My Orders Screen

Two-tab layout separating **Ongoing** and **History** orders.

Interactions:

- Tab switching filters the order list

- Swipe left on order card → Toggle status between ongoing and history

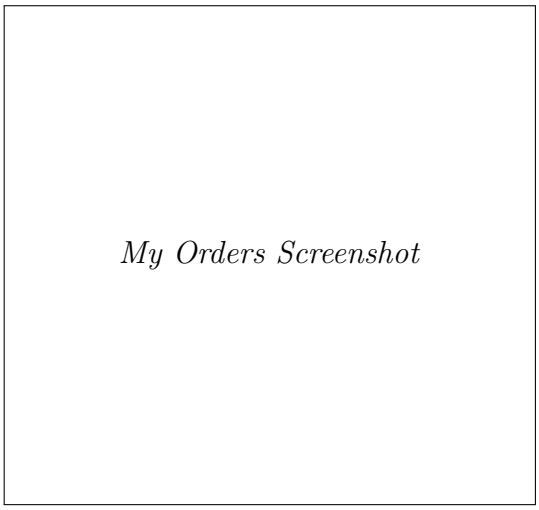


Figure 6: My Orders Screen

3.6 Rewards Screen

Displays the loyalty card with current stamp count, total reward points, and transaction history.

Interactions:

- Tap loyalty card at 8 stamps → Reset to 0
- Redeem button → Navigate to Redeem Rewards Screen

Business rules:

- +1 stamp per completed order, maximum 8
- Points awarded: $\lfloor \text{orderTotal}/10{,}000 \rfloor$

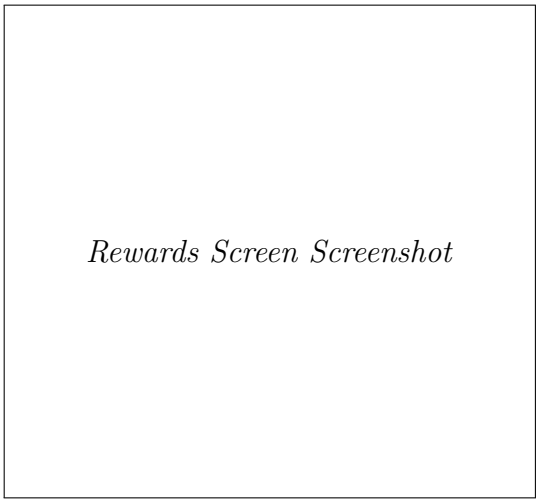


Figure 7: Rewards Screen

3.7 Redeem Rewards Screen

Product catalog showing items available for point redemption. Tapping an item shows a confirmation dialog; confirming deducts points from the balance.

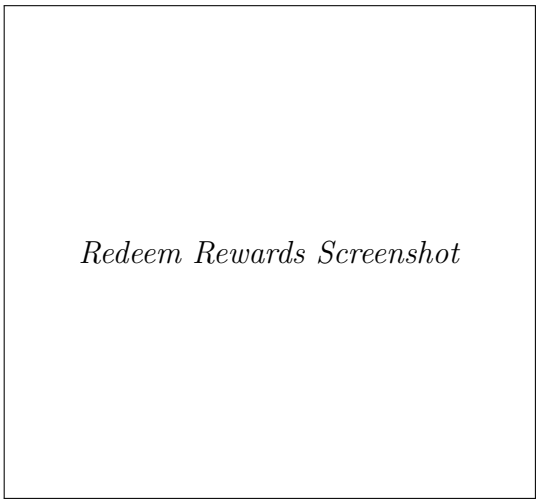


Figure 8: Redeem Rewards Screen

3.8 Profile Screen

Displays user profile (name, email, phone, address, avatar). An edit icon toggles inline editing; changes are persisted via `Hive`.

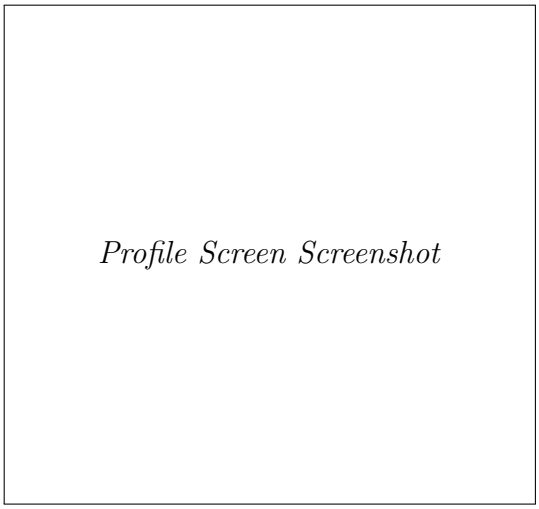


Figure 9: Profile Screen

4 Domain Model

4.1 Entity Overview

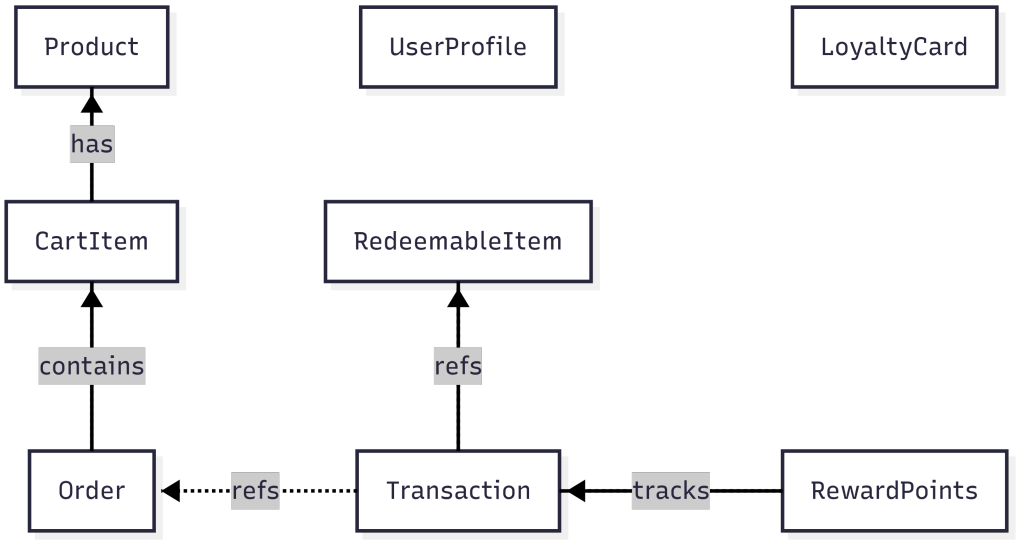


Figure 10: Entity Overview

4.2 Entity Details

Entity	Key Attributes	Business Logic
Product	id, name, basePrice, availableCustomizations	Unit price = basePrice + sum of selected customization deltas
CartItem	product, selectedCustomizations, quantity	`calculateSubtotal()` = unit price × quantity
Order	id, items, status, createdAt	`calculateTotalPrice()` = $\sum$ item subtotals `calculatePointsEarned()` = total ÷ 10,000
LoyaltyCard	stampCount (0–8)	Increment capped at 8; reset only when exactly 8
RewardPoints	balance, transactions	`calculateTotalEarned()` = sum of earn transactions Redemption rejected if insufficient balance
Transaction	type (earn/redeem), amount, createdAt	References orderId or redeemableItemId
RedeemableItem	id, name, pointCost	Catalog item for point redemption
UserProfile	name, email, phone, avatarUrl	• Name must be non-empty • Email validated by regex • Phone (optional) must match VN format `0[35789]\d{8}`

Table 2: Entity Details

5 Business Rules Implementation

1. **Loyalty stamps:** +1 per completed order, capped at 8. At 8 stamps, the user may tap the loyalty card to reset to 0. Enforced in `LoyaltyCard` entity with `assert` guards.
2. **Reward points:** $\lfloor \text{orderTotal}/10{,}000 \rfloor$ points per order. Points are redeemable for products from the catalog. Insufficient balance blocks redemption.
3. **Order lifecycle:** Orders are created with "ongoing" status. The user can swipe to move an order to "history" (and back).
4. **Cart persistence:** Items persist with their customizations until explicitly removed or checked out.
5. **Dynamic pricing:** Total updates reactively when quantity or customization options change, computed via `CartItem.calculateSubtotal()`.

6. **Data persistence:** User profile, orders, cart items, and loyalty stamp count are persisted via `Hive`. Seed data provides initial values; persisted values override on subsequent launches.

6 Testing

All tests use the built-in `flutter_test` framework with no external testing dependencies.

6.1 Test Summary

Category	**Files**	**Tests**
Unit tests	7	68
Integration tests	1	11
UI (widget) tests	11	72
Total	**19**	**151**

6.2 Unit Tests

Unit tests cover all domain entities and business logic in isolation, using pure Dart with no widget dependencies.

6.2.1 Entity Tests

Entity	**Test Cases**
Product	• Create with valid name, price, customizations • Calculate unit price from base price • Calculate unit price with customization adjustments • Edge: zero or negative base price • Edge: name with max length or special characters
CartItem	• Create with product and selected customizations • Update quantity; verify subtotal recalculates • Edge: quantity zero or negative • Edge: no customizations selected
Order	• Create from cart items with correct total • Toggle status ongoing $\leftrightarrow$ history • Timestamp recorded correctly • Edge: empty items list • Edge: idempotent toggle

Entity	Test Cases
LoyaltyCard	• Start at 0 stamps; increment up to max 8 • Edge: increment beyond 8 (no change) • Edge: reset at exactly 8; no reset below 8
RewardPoints	• Award points by order value • Deduct on redemption; track earn/redeem transactions • `calculateTotalEarned()` sums earn transactions • Edge: insufficient balance rejection • Edge: zero/negative point redemption
RedeemableItem	• Create with valid name and point cost • Edge: zero or negative point cost
Transaction	• Create earn/redeem transactions with correct fields • `createdAt` auto-set • Edge: zero or negative amount
UserProfile	• Create and update name, email, phone, avatar • Edge: empty fields, invalid email format

6.2.2 Business Logic Tests

Logic Area	Test Cases
Dynamic Pricing	• Total from base price alone • Total with customization deltas • Multiply by quantity; real-time update on option change • Edge: zero quantity → zero total
Cart Total	• Single item total; aggregate for multiple items • Recalculate after item removal • Edge: empty cart → total of 0
Points Calc.	• Correct points from order value • `calculateTotalPrice()` and `calculatePointsEarned()` consistency • Edge: zero-value order; large value overflow
Points Aggregation	• Sum from multiple earn transactions • Partial redemption; balance tracking • Edge: zero balance; repeated redemptions
Loyalty Stamps	• +1 per order; stops at max 8 • Reset at 8; first-order increment

6.3 Integration Tests

Integration tests exercise real workflows spanning multiple entities without mocking:

Workflow	Scenarios
Add to Cart → Checkout → Order	Add items, calculate totals, modify cart, empty cart rejected, order lifecycle
Order → Stamps → Points	Award stamps & points on completion, 8-stamp reset, redeem & deduct, insufficient balance
Loyalty Card Reset	Reset only at 8 stamps, no-op below 8

6.4 UI (Widget) Tests

Widget tests verify screen rendering and user interactions with mocked ViewModels.

Screen / Interaction	Test Cases
Home Screen	Header display, loyalty card rendering, product grid, bottom navigation
Details Screen	Product info display, customization controls, add to cart with centered overlay confirmation, cart icon navigation
Cart Screen	Cart item list, swipe-to-remove, total recalculation, checkout flow, empty cart guard
Orders Screen	Two-tab layout, ongoing/history filtering, swipe to toggle status, empty state
Rewards Screen	Points balance, loyalty card, Redeem button, empty transaction history
Redeem Rewards	Product catalog, confirmation dialog, point deduction, disabled when insufficient
Profile Screen	Profile display, edit mode toggle, save changes, cancel discards, field isolation

7 Conclusion

The Code Cup implements all 8 specified screens with complete business logic for ordering, loyalty, rewards, and profile management. The MVVM architecture keeps domain logic testable and UI code thin. With 151 passing tests (68 unit + 11 integration + 72 widget), all entity invariants and screen interactions are verified.